

07 — Orchestration Service: Core Modules (Deep Dive)

Module purpose: SAP AI Core’s managed pipeline layer — one declarative `OrchestrationConfig` runs templating, safety, and the LLM call **server-side**, replacing hand-written Python glue (contrast with sheet 02’s proxy: one model, one call, no pipeline). Five files.

File 1 — `orchestration_setup.py`

```
from gen_ai_hub.orchestration.service import OrchestrationService

def get_orchestration_service(api_url: str = None) -> OrchestrationService:
    api_url = api_url or os.getenv("ORCHESTRATION_DEPLOYMENT_URL")
    if api_url:
        service = OrchestrationService(api_url=api_url)
    else:
        service = OrchestrationService()      # SDK auto-discovers the newest RUNNING
        ↪ orchestration deployment
    return service
```

Working explanation: an **orchestration deployment** is a different beast from a plain model deployment (sheet 02) — it’s created against the orchestration **scenario** in AI Core, and what model(s) it actually calls is decided per-request inside your `OrchestrationConfig`, not fixed at deployment time. Credential resolution (env vars → `~/.aicore/config.json` → `VCAP_SERVICES`) happens transparently the same way as sheet 02’s `get_proxy_client`.

File 2 — `templating_module.py`

```
from gen_ai_hub.orchestration.models.template import Template, TemplateValue
from gen_ai_hub.orchestration.models.message import SystemMessage, UserMessage,
    ↪ AssistantMessage

def build_template(system_prompt: str, user_prompt: str = "{{?text}}") -> Template:
    return Template(messages=[SystemMessage(system_prompt), UserMessage(user_prompt)])

def build_few_shot_template(system_prompt: str, examples: list[tuple[str, str]]) -> Template:
    messages = [SystemMessage(system_prompt)]
    for user_text, assistant_text in examples:                # example PAIRS precede the
        ↪ real question
        messages += [UserMessage(user_text), AssistantMessage(assistant_text)]
    messages.append(UserMessage("{{?text}}"))                # the real (variable)
    ↪ question goes last
    return Template(messages=messages)
```

Working explanation: `{{?name}}` is orchestration’s placeholder syntax — substituted **server-side, after masking, before the LLM call** (this ordering matters — see File 5). Few-shot prompting works by literally inserting example Q/A pairs as prior “turns” before the real question — the model conditions on them as conversational context, which tends to steer output format/style more reliably than describing the desired format in prose alone.

File 3 — `llm_module.py`

```
from gen_ai_hub.orchestration.models.llm import LLM

def build_llm(model="gpt-4o-mini", version="latest", temperature=0.0, max_tokens=500,
```

```

        top_p=1.0, frequency_penalty=0.0, presence_penalty=0.0) -> LLM:
    return LLM(name=model, version=version, parameters={
        "max_tokens": max_tokens, "temperature": temperature, "top_p": top_p,
        "frequency_penalty": frequency_penalty, "presence_penalty": presence_penalty,
    })

```

Working explanation: swapping providers (OpenAI → Gemini → Claude on AI Core) is purely a name= string change — the harmonized API means every provider is called through this exact same class shape. temperature and top_p are two DIFFERENT randomness controls — convention is to tune only one at a time (temperature scales the whole probability distribution; top_p truncates to the smallest token set covering probability mass p). frequency_penalty discourages repeating the same tokens; presence_penalty discourages repeating the same topics — subtly different despite sounding similar.

File 4 — content_filtering_module.py

```

from gen_ai_hub.orchestration.models.content_filter import AzureContentFilter
from gen_ai_hub.orchestration.models.filtering import Filtering, InputFiltering,
↳ OutputFiltering

STRICT_FILTER = AzureContentFilter(hate=0, sexual=0, self_harm=0, violence=0)  # 0 = block
↳ almost everything
RELAXED_FILTER = AzureContentFilter(hate=4, sexual=4, self_harm=4, violence=4)  # 4 = allow
↳ up to "medium" severity

def build_filtering(input_filter=None, output_filter=None) -> Filtering:
    kwargs = {}
    if input_filter is not None: kwargs["input_filtering"] =
    ↳ InputFiltering(filters=[input_filter])
    if output_filter is not None: kwargs["output_filtering"] =
    ↳ OutputFiltering(filters=[output_filter])
    return Filtering(**kwargs)  # either side is optional – pass only what
    ↳ you need

```

Working explanation: each severity number is the **highest severity you're willing to ALLOW through** per category — content scored above your threshold triggers a block (raised as an exception on .run()). Input filtering runs BEFORE the LLM call (blocks abusive prompts, saving cost/latency on a call that would've been rejected anyway); output filtering runs AFTER (catches a model producing unsafe content even from an innocuous-looking prompt) — the two catch genuinely different failure modes, which is why both exist as independent, composable options.

File 5 — data_masking_module.py

```

from gen_ai_hub.orchestration.models.data_masking import DataMasking
from gen_ai_hub.orchestration.models.sap_data_privacy_integration import
↳ SAPDataPrivacyIntegration, MaskingMethod, ProfileEntity

def build_masking(method=MaskingMethod.PSEUDONYMIZATION, entities=None, allowlist=None) ->
↳ DataMasking:
    entities = entities or [ProfileEntity.PERSON, ProfileEntity.EMAIL, ProfileEntity.PHONE,
    ↳ ProfileEntity.ORG]

```

```
return DataMasking(providers=[SAPDataPrivacyIntegration(method=method,
↳ entities=entities, allowlist=allowlist or []]))
```

Working explanation: anonymization replaces a detected entity with a generic label (“John Smith” → “PERSON_1”) — one-way, the original is discarded, used when the LLM truly never needs the real value. **Pseudonymization** replaces it with a consistent fake value FOR THE DURATION OF THAT CALL, and orchestration automatically restores the real value in the final response your app receives — the LLM/provider’s servers/logs never see the real PII, but your app still gets a usable answer with real names/emails filled back in. `allowlist` exists because masking classifiers can over-match — e.g. your own product name might pattern-match as an ORG entity and get masked unnecessarily.

Full Config + Run (tying files 1-5 together)

```
from gen_ai_hub.orchestration.models.config import OrchestrationConfig

config = OrchestrationConfig(template=template, llm=llm, filtering=filtering,
↳ data_masking=masking)
result = service.run(config=config, template_values=[TemplateValue(name="text",
↳ value="Hi!")])
print(result.orchestration_result.choices[0].message.content)
```

Execution order inside one `.run()` call: masking → input filtering → templating (placeholder substitution) → (grounding, sheet 08) → LLM → output filtering → un-masking. This order is itself a common interview question — masking must run before the LLM sees the text (PII never reaches the model), and un-masking must run after the LLM responds.

Key Points

- An orchestration **deployment** ≠ a model **deployment** — the model is chosen per-request inside the config, not fixed at deploy time.
- `{{?name}}` placeholders are substituted server-side, AFTER masking, BEFORE the LLM call.
- Every module (`llm`, `filtering`, `data_masking`, and later `grounding`) is **optional and additive** to `OrchestrationConfig` — only `template` + `llm` are required.
- Content filter severity = the max severity you ALLOW; masking method choice (anonymize vs. pseudonymize) determines whether the real value comes back in the response.
- temperature vs. `top_p`: tune one, not both, for controlling randomness.

Interview Q&A

Q: Orchestration Service vs. Gen AI Hub proxy (sheet 02) — when do you pick which?

A: Proxy for a single model call with no governance overhead. Orchestration when you need masking/filtering/grounding applied consistently and auditably across many services, configured once rather than hand-coded into every caller.

Q: Anonymization vs pseudonymization — what’s the real, testable difference?

A: Anonymization is irreversible (generic label, original discarded). Pseudonymization is reversible within that single call — the model sees a fake stand-in, but the FINAL RESPONSE your app receives has the real value automatically restored by the orchestration pipeline.

Q: Why does masking have to run before the LLM call and un-masking after — could you swap the order?

A: No — the whole point is the LLM/provider never sees the real value. Masking before ensures raw PII never leaves your security boundary; un-masking after is what makes the response usable by your app/user without them having to manually resolve placeholder tokens.

Q: What do the AzureContentFilter severity numbers (0/2/4/6) actually represent? A: They're threshold values on Azure Content Safety's 0-6+ severity scale per category — your configured number is the highest severity you tolerate; anything scored higher triggers a block.